

GUÍA DE CLASE PARA EL DOCENTE (LESSON PLAN)

Código: IMT-342-GP-0202

Unidad I: Morfología, Geometría del Espacio y Cinemática Directa

Tema: Parametrización Mínima: Ángulos de Euler (ZYZ), RPY, Gimbal Lock e Intro a Cuaterniones

Duración: 140 minutos reales (Jueves 13 de Agosto, 16:00 a 17:45)

Alineamiento: CDIO (Diseñar/Implementar) + ABET SO-1 (Modelado Analítico) y SO-6 (Experimentación)

Estructura de la Sesión y Gestión del Tiempo (140 min)

Tiempo	Fase	Actividad Docente y Estudiantil	Justificación Pedagógica
16:00 - 16:15	Activación y Anclaje (15 min)	Demostración física: Giros-copio o 3 anillos 3D. Alinear dos ejes para mostrar pérdida de GDL.	Visualización Concreta: El Gimbal Lock es un fenómeno físico real, no solo una falla algebraica.
16:15 - 17:00	Deducción ZYZ y RPY (45 min)	Deducción en pizarra de $R_{ZYZ}(\phi, \theta, \psi)$ y $R_{RPY}(\gamma, \beta, \alpha)$.	Rigor de Código: Entender la traducción de ángulos a matrices en controladores CAD/CAM.
17:00 - 17:25	Análisis Gimbal Lock (25 min)	Evaluar R_{ZYZ} con $\theta = 0$. Demostrar el acople $(\phi + \psi)$.	Conexión PFI (ABB IRB 120): La postura erguida de la muñeca ($q_5 = 0$) causa la singularidad.
17:25 - 17:40	Cuaterniones (15 min)	Explicar por qué ROS exige Cuaterniones unitarios ($\mathbf{q} \in S^3$) para evitar singularidades.	Perspectiva Software: Justifica el tipo de mensaje <code>geometry_msgs/Quaternion</code> en ROS.
17:40 - 17:45	Cierre y Asignación (5 min)	Taller SymPy/Python para Guía TP N° 2.	Alineamiento: Evidencia bajo validación numérico-simbólica.

Código Maestro de Verificación (Referencia Docente)

Este script utiliza SymPy para validar analíticamente la pérdida de un grado de libertad representacional.

```

1 import sympy as sp
2 import numpy as np
3
4 def verify_gimbal_lock_symbolic():
5     print("== VERIFICACION SIMBOLICA DE GIMBAL LOCK (ZYZ) ==")
6     phi, theta, psi = sp.symbols('phi theta psi', real=True)
7
8     # Matrices de rotacion canonicas simbolicas
9     Rz1 = sp.Matrix([[sp.cos(phi), -sp.sin(phi), 0],
10                     [sp.sin(phi), sp.cos(phi), 0],
11                     [0, 0, 1]])
12     Ry = sp.Matrix([[sp.cos(theta), 0, sp.sin(theta)],
13                    [0, 1, 0],
14                    [-sp.sin(theta), 0, sp.cos(theta)]])
15     Rz2 = sp.Matrix([[sp.cos(psi), -sp.sin(psi), 0],
16                     [sp.sin(psi), sp.cos(psi), 0],
17                     [0, 0, 1]])
18
19     # Matriz Euler ZYZ y Evaluacion en theta = 0
20     R_zyz = Rz1 * Ry * Rz2
21     R_singular = sp.simplify(R_zyz.subs(theta, 0))
22     print("\nMatriz R_ZYZ en theta = 0:")
23     sp.pprint(R_singular)
24
25     # Demostrar dependencia de (phi + psi)
26     alpha = sp.symbols('alpha')
27     R_expected = sp.Matrix([[sp.cos(alpha), -sp.sin(alpha), 0],
28                             [sp.sin(alpha), sp.cos(alpha), 0],
29                             [0, 0, 1]])
30     print("\nEs equivalente a Rz(phi + psi)?:",
31           R_singular == R_expected.subs(alpha, phi + psi))
32
33 if __name__ == "__main__":
34     verify_gimbal_lock_symbolic()

```

UNIVERSIDAD CATÓLICA BOLIVIANA "SAN PABLO"

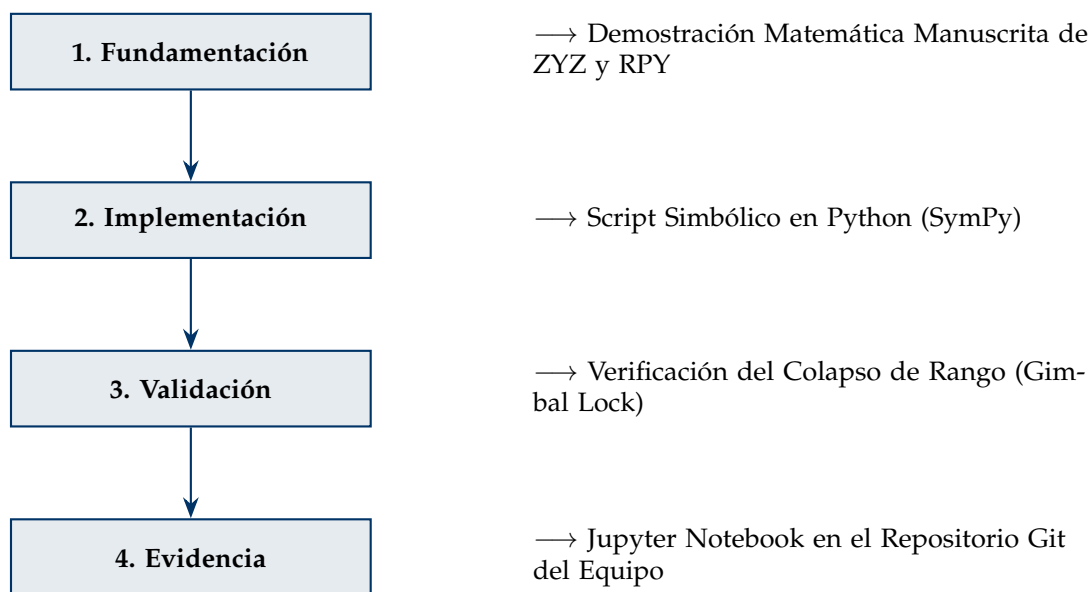
SEDE TARIJA

Departamento de Ciencias Exactas e Ingenierías | Carrera de Ingeniería Mecatrónica
Asignatura: Robótica (IMT-342) | Gestión: II-2026

Hoja de Trabajo Práctico N° 2

Ángulos de Euler, Gimbal Lock y Validación Simbólica/Numérica

1. Principios Obligatorios de la Práctica (CDIO)



2. Ejercicios de Desarrollo Obligatorio

Ejercicio 1: Deducción Simbólica y Extracción (Teoría)

- Demuestre analíticamente en papel que la secuencia de ejes fijos Roll-Pitch-Yaw $R_z(\gamma)R_y(\beta)R_x(\alpha)$ produce exactamente la misma matriz de rotación que la secuencia de ejes móviles $X \rightarrow Y' \rightarrow Z''$ con ángulos (α, β, γ) .
- Desarrolle el sistema de ecuaciones para extraer analíticamente los ángulos α, β, γ a partir de los componentes r_{ij} de una matriz $R \in SO(3)$ conocida, utilizando la función trigonométrica no ambigua $\text{atan2}(y, x)$.

Ejercicio 2: Análisis del Gimbal Lock en SymPy (Implementación)

Escriba un script en Python utilizando la librería de matemática simbólica SymPy que:

- Declare las variables simbólicas `phi, theta, psi = sp.symbols('phi theta psi')`.
- Defina las matrices de rotación simbólicas $R_z(\phi)$, $R_y(\theta)$ y $R_z(\psi)$ y multiplíquelas para obtener R_{ZYZ} .

3. Sustituya el valor $\theta = 0$ en la matriz resultante mediante `.subs(theta, 0)` y aplique `sp.simplify()`.
4. Demuestre por código simbólico que las derivadas parciales respecto a ϕ y ψ se vuelven linealmente dependientes:

$$\left. \frac{\partial R_{ZYZ}}{\partial \phi} \right|_{\theta=0} = \left. \frac{\partial R_{ZYZ}}{\partial \psi} \right|_{\theta=0}$$

Ejercicio 3: Simulación Caso de Estudio ABB IRB 120 (Validación)

Un usuario comanda al robot ABB IRB 120 a adoptar una orientación en su muñeca dada por los ángulos de Euler ZYZ: $\phi = 45^\circ$, $\theta = 0,0001^\circ$ (casi cero) y $\psi = 30^\circ$.

1. Utilizando NumPy, evalúe numéricamente la matriz R_{ZYZ} .
2. Utilice sus ecuaciones del Ejercicio 1 para intentar recuperar los ángulos (ϕ, θ, ψ) a partir de la matriz calculada.
3. Repita el cálculo para $\theta = 0,0000000001^\circ$ y analice la inestabilidad de los ángulos recuperados debido al truncamiento de punto flotante.
4. Grafique en el Notebook el error de extracción angular $|\phi_{\text{real}} - \phi_{\text{calculado}}|$ en función de θ cuando $\theta \rightarrow 0$.